

Skylight—A Window on Shingled Disk Operation

ABUTALIB AGHAYEV, MANSOUR SHAF AEI, and PETER DESNOYERS,
Northeastern University

We introduce SkyLight, a novel methodology that combines software and hardware techniques to reverse engineer key properties of drive-managed Shingled Magnetic Recording (SMR) drives. The software part of SkyLight measures the latency of controlled I/O operations to infer important properties of drive-managed SMR, including type, structure, and size of the persistent cache; type of cleaning algorithm; type of block mapping; and size of bands. The hardware part of SkyLight tracks drive head movements during these tests, using a high-speed camera through an observation window drilled through the cover of the drive. These observations not only confirm inferences from measurements, but resolve ambiguities that arise from the use of latency measurements alone. We show the generality and efficacy of our techniques by running them on top of three emulated and two real SMR drives, discovering valuable performance-relevant details of the behavior of the real SMR drives.

Categories and Subject Descriptors: C.4 [Performance of Systems]: Design studies; Measurement techniques; Modeling techniques; Performance attributes; D.4.2 [Storage Management]: Allocation/deallocation strategies; Garbage collection; Secondary storage

General Terms: Design, Algorithms, Experimentation, Measurement, Performance, Reliability

Additional Key Words and Phrases: Shingled magnetic recording, shingle translation layer, emulation, microbenchmarks, disks

ACM Reference Format:

Abutalib Aghayev, Mansour Shafaei, and Peter Desnoyers. 2015. SkyLight—A window on shingled disk operation. ACM Trans. Storage 11, 4, Article 16 (October 2015), 28 pages.

DOI: <http://dx.doi.org/10.1145/2821511>

1. INTRODUCTION

In the nearly 60 years since the Hard Disk Drive (HDD) has been introduced, it has become the mainstay of computer storage systems. In 2013 the hard drive industry shipped over 400EB [Seagate 2013b] of storage, or almost 60GB for every person on earth. Although facing strong competition from NAND flash-based Solid-State Drives (SSDs), magnetic disks hold a $10\times$ advantage over flash in both total bits shipped [Riley 2013] and per-bit cost [DRAMeXchange 2014], an advantage that will persist if density improvements continue at current rates.

The most recent growth in disk capacity is the result of improvements to Perpendicular Magnetic Recording (PMR) [Piramanayagam 2007], which has yielded terabyte drives by enabling bits as short as 20nm in tracks 70nm wide [Seagate 2013c], but

This work was supported by the National Science Foundation, under grant CNS-1149232, and by NetApp Faculty Fellowship.

Authors' addresses: A. Aghayev, M. Shafaei, and P. Desnoyers, College of Computer and Information Science, Northeastern University, 440 Huntington Ave., 202 West Village H, Boston, MA; emails: agayev@gmail.com, shafaei@ece.neu.edu, pjd@ccs.neu.edu.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.

© 2015 ACM 1553-3077/2015/10-ART16 \$15.00

DOI: <http://dx.doi.org/10.1145/2821511>

further increases will require new technologies [Thompson and Best 2000]. Shingled Magnetic Recording (SMR) [Wood et al. 2009] is the first such technology to reach market: 5TB drives are available from Seagate [2013a] and shipments of 8 and 10TB drives have been announced by Seagate [2014] and HGST [2014]. Other technologies (Heat-Assisted Magnetic Recording [Kryder et al. 2008] and Bit-Patterned Media [Dobisz et al. 2008]) remain in the research stage, and may in fact use shingled recording when they are released [Wang et al. 2013].

Shingled recording spaces tracks more closely, so they overlap like rows of shingles on a roof, squeezing more tracks and bits onto each platter [Wood et al. 2009]. The increase in density comes at a cost in complexity, as modifying a disk sector will corrupt other data on the overlapped tracks, requiring copying to avoid data loss [Amer et al. 2010; Feldman and Gibson 2013; Gibson and Ganger 2011; Gibson and Polte 2009]. Rather than push this work onto the host file system [INCITS T10 Technical Committee 2014; Le Moal et al. 2012], SMR drives shipped to date preserve compatibility with existing drives by implementing a Shingle Translation Layer (STL) [Cassuto et al. 2010; Gibson and Ganger 2011; Hall et al. 2012] that hides this complexity.

Like an SSD, an SMR drive combines out-of-place writes with dynamic mapping in order to efficiently update data, resulting in a drive with performance much different from that of a Conventional Magnetic Recording (CMR) drive due to seek overhead for out-of-order operations. However, unlike SSDs, which have been extensively measured and characterized [Bouganim et al. 2009; Chen et al. 2009], little is known about the behavior and performance of SMR drives and their translation layers, or how to optimize file systems, storage arrays, and applications to best use them.

We introduce a methodology for measuring and characterizing such drives, developing a specific series of microbenchmarks for this characterization process, much as has been done in the past for conventional drives [Gim and Won 2010; Talagala et al. 1999; Worthington et al. 1995]. We augment these timing measurements with a novel technique that tracks actual head movements via high-speed camera and image processing and provides a source of reliable information in cases where timing results are ambiguous.

We validate this methodology on three different emulated drives that use STLs previously described in the literature [Cassuto et al. 2010; Coker and Hall 2013; Hall et al. 2012], implemented as a Linux *device mapper target* [Device-Mapper 2001] over a conventional drive, demonstrating accurate inference of properties. We then apply this methodology to 5 and 8TB SMR drives provided by Seagate, inferring the STL algorithm and its properties and providing the first public characterization of such drives.

Using our approach we are able to discover important characteristics of the Seagate SMR drives and their translation layer, including the following:

- Cache type and size.* The drives use a persistent disk cache of 20 and 25GiB on the 5 and 8TB drives, respectively, with high random write speed until the cache is full. The effective cache size is a function of write size and queue depth.
- Persistent cache structure.* The persistent disk cache is written as journal entries with quantized sizes—a phenomenon absent from the academic literature on SMRs.
- Block mapping.* Noncached data is statically mapped, using a fixed assignment of Logical Block Addresses (LBAs) to Physical Block Addresses (PBAs), similar to that used in CMR drives, with implications for performance and durability.
- Band size.* SMR drives organize data in *bands*—a set of contiguous tracks that are rewritten as a unit; the examined drives have a small band size of 15–40MiB.
- Cleaning mechanism.* Aggressive cleaning during idle times moves data from the persistent cache to bands; cleaning duration is 0.6–1.6s per modified band.

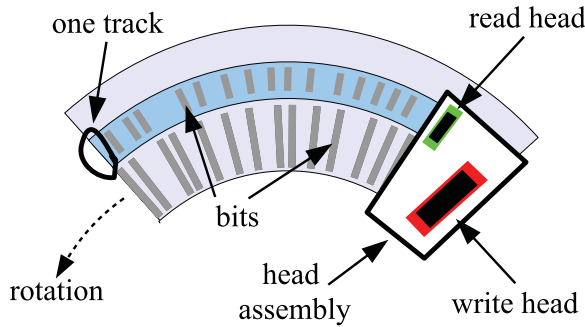


Fig. 1. Shingled disk tracks with head width $k = 2$.

Our results show the details that may be discovered using Skylight, most of which impact (negatively or positively) the performance of different workloads, as described in Section 6. These results—and the toolset allowing similar measurements on new drives—should thus be useful to users of SMR drives, both in determining what workloads are best suited for these drives and in modifying applications to better use them. In addition, we hope that they will be of use to designers of SMR drives and their translation layers, by illustrating the effects of low-level design decisions on system-level performance.

In the rest of the article we give an overview of SMR (Section 2) followed by the description of emulated and real drives examined (Section 3). We then present our characterization methodology and apply it to all of the drives (Section 4); finally, we survey related work (Section 5) and present our conclusions (Section 6).

2. BACKGROUND

Shingled recording is a response to limitations on areal density with perpendicular magnetic recording due to the superparamagnetic limit [Thompson and Best 2000]. In brief, for bits to become smaller, write heads must become narrower, resulting in weaker magnetic fields. This requires lower coercivity (easily recordable) media, which is more vulnerable to bit flips due to thermal noise, requiring larger bits for reliability. As the head gets smaller this minimum bit size gets larger, until it reaches the width of the head and further scaling is impossible.

Several technologies have been proposed to go beyond this limit, of which SMR is the simplest [Wood et al. 2009]. To decrease the bit size further, SMR reduces the track width while keeping the head size constant, resulting in a head that writes a path several tracks wide. Tracks are then overlapped like rows of shingles on a roof, as seen in Figure 1. Writing these overlapping tracks requires only incremental changes in manufacturing, but much greater system changes, as it becomes impossible to rewrite a single sector without destroying data on the overlapped sectors.

For maximum capacity an SMR drive could be written from beginning to end, utilizing all tracks. Modifying any of this data, however, would require reading and rewriting the data that would be damaged by that write, and data to be damaged by the rewrite and so on, until the end of the surface is reached. This cascade of copying may be halted by inserting *guard regions*—tracks written at the full head width—so that the tracks before the guard region may be rewritten without affecting any tracks following it, as shown in Figure 2. These guard regions divide each disk surface into rewritable bands;

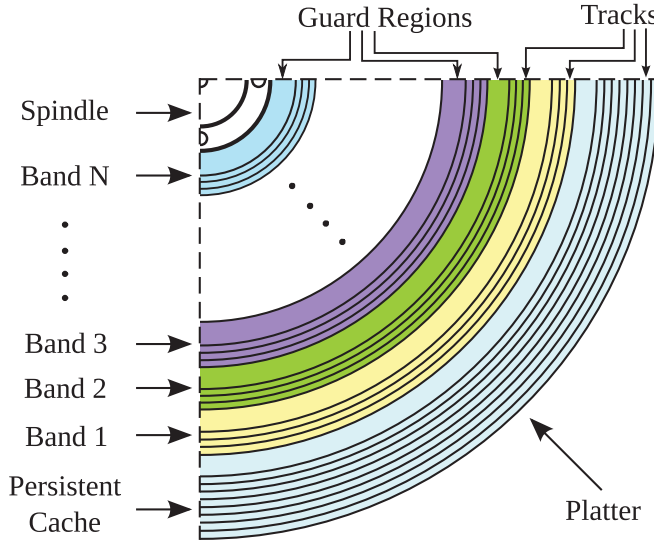


Fig. 2. Surface of a platter in a hypothetical SMR drive. A persistent cache consisting of nine tracks is located at the outer diameter. The guard region that separates the persistent cache from the first band is simply a track that is written at a full head width of k tracks. Although the guard region occupies the width of k tracks, it contains a single track's worth of data and the remaining $k-1$ tracks are wasted. The bands consist of four tracks, also separated with a guard region. Overwriting a sector in the last track of any band will not affect the following band. Overwriting a sector in any of the tracks will require reading and rewriting all of the tracks starting at the affected track and ending at the guard region within the band.

since the guards hold a single track's worth of data, storage efficiency for a band size of b tracks is $\frac{b}{b+k-1}$.

Given knowledge of these bands, a host file system can ensure they are only written sequentially, for example, by implementing a log-structured file system [Le Moal et al. 2012; Rosenblum and Ousterhout 1991]. Standards are being developed to allow a drive to identify these bands to the host [INCITS T10 Technical Committee 2014]: *host-aware* drives report sequential-write-preferred bands (an internal STL handles nonsequential writes), and *host-managed* drives report sequential-write-required bands. These standards are still in draft form, and to date no drives based on them are available on the open market.

Alternately the *drive-managed* disks present a standard rewritable block interface that is implemented by an internal STL, much as an SSD uses a Flash Translation Layer (FTL). Although the two are logically similar, appropriate algorithms differ due to differences in the constraints placed by the underlying media: (a) high seek times for nonsequential access, (b) lack of high-speed reads, (c) use of large (tens to hundreds of MB) cleaning units, and (d) lack of wear-out, eliminating the need for wear leveling.

These translation layers typically store all data in bands where it is mapped at a coarse granularity, and devote a small fraction of the disk to a *persistent cache*, as shown in Figure 2, which contains copies of recently written data. Data that should be retrieved from the persistent cache may be identified by checking a *persistent cache map* (or *exception map*) [Cassuto et al. 2010; Hall et al. 2012]. Data is moved back from the persistent cache to bands by the process of *cleaning*, which performs Read-Modify-Write (RMW) on every band whose data was overwritten. The cleaning process may be *lazy*, running only when the free cache space is low, or *aggressive*, running during idle times.

In one translation approach, a static mapping algorithmically assigns a *native location* [Cassuto et al. 2010] (a PBA) to each LBA in the same way as is done in a CMR drive. An alternate approach uses coarse-grained dynamic mapping for noncached LBAs [Cassuto et al. 2010], in combination with a small number of free bands. During cleaning, the drive writes an updated band to one of these free bands and then updates the dynamic map, potentially eliminating the need for a temporary staging area for cleaning updates and sequential writes.

In any of these cases drive operation may change based on the setting of the *volatile cache* (enabled or disabled) [SATA-IO 2011]. When the volatile cache is disabled, writes are required to be persistent before completion is reported to the host. When it is enabled, persistence is only guaranteed after a FLUSH command or a write command with the flush (FUA) flag set.

3. TEST DRIVES

We now describe the drives we study. First, we discuss how we emulate three SMR drives using our implementation of two STLs described in the literature. Second, we describe the real SMR drives we study in this article and the real CMR drive we use for emulating SMR drives.

3.1. Emulated Drives

We implement Cassuto et al.'s *set-associative* STL [Cassuto et al. 2010] and a variant of their *S-blocks* STL [Cassuto et al. 2010; Hall 2014], which we call *fully associative* STL, as Linux device mapper targets. These are kernel modules that export a pseudo block device to user space that internally behaves like a drive-managed SMR—the module translates incoming requests using the translation algorithm and executes them on a CMR drive.

The *set-associative* STL manages the disk as a set of N isocapacity (same-sized) data bands, with typical sizes of 20–40MiB, and uses a small (1%–10%) section of the disk as the persistent cache. The persistent cache is also managed as a set of n isocapacity cache bands where $n \ll N$. When a block in data band a is to be written, a cache band is chosen through $(a \bmod n)$; the next empty block in this cache band is written and the persistent cache map is updated. Further accesses to the block are served from the cache band until cleaning moves the block to its native location, which happens when the cache band becomes full.

The *fully associative* STL, on the other hand, divides the disk into large (we used 40GiB) zones and manages each zone independently. A zone starts with 5% of its capacity provisioned to free bands for handling updates. When a block in logical band a is to be written to the corresponding physical band b , a free band c is chosen and written to and the persistent cache map is updated. When the number of free bands falls below a threshold, cleaning merges the bands b and c and writes it to a new band d and remaps the logical band a to the physical band d , freeing bands b and c in the process. This dynamic mapping of bands allows the fully associative STL to handle streaming writes with zero overhead.

To evaluate the accuracy of our emulation strategy, we implemented a pass-through device mapper target and found negligible overhead for our tests, confirming a previous study [Pitchumani et al. 2012]. Although in theory, this emulation approach may seem disadvantaged by the lack of access to exact sector layout, in practice this is not the case—even in real SMR drives, the STL running inside the drive is implemented on top of a layer that provides linear PBAs by hiding sector layout and defect management [Feldman 2014b]. Therefore, we believe that the device mapper target running on top of a CMR drive provides an accurate model for predicting the behavior of an STL implemented by the controller of an SMR drive.

Table I. Emulated SMR Drive Configurations

Drive Name	STL	Persistent Cache Type and Size	Disk Cache Multiplicity	Cleaning Type	Band Size	Mapping Type	Size
Emulated-SMR-1	Set associative	Disk, 37.2GiB	Single at ID	Lazy	40MiB	Static	3.9TB
Emulated-SMR-2	Set associative	Flash, 9.98GiB	N/A	Lazy	25MiB	Static	3.9TB
Emulated-SMR-3	Fully associative	Disk, 37.2GiB	Multiple	Aggressive	20MiB	Dynamic	3.9TB

Table I shows the three emulated SMR drive configurations we use in our tests. The first two drives use the set-associative STL, and they differ in the type of persistent cache and band size. The last drive uses the fully associative STL and disk for the persistent cache. We do not have a drive configuration combining the fully associative STL and flash for persistent cache, since the fully associative STL aims to reduce long seeks during cleaning in disks, by using multiple caches evenly spread out on a disk—flash does not suffer from long seek times.

To emulate an SMR drive with a flash cache (Emulate-SMR-2) we use the Emulate-SMR-1 implementation, but use a device mapper linear target to redirect the underlying LBAs corresponding to the persistent cache, storing them on an SSD.

To check the correctness of the emulated SMR drives we ran repeated burn-in tests using `fio` [Axboe 2015]. We also formatted emulated drives with `ext4`, compiled the Linux kernel on top, and successfully booted the system with the compiled kernel. The source code for the set-associative STL (1,200 lines of C) and a testing framework (250 lines of Go) are available at <http://sssl.ccs.neu.edu/skylight>.

3.2. Real Drives

Two real SMR drives were tested: Seagate ST5000AS0011, a 5,900RPM desktop drive (rotation time ≈ 10 ms) with four platters, eight heads, and 5TB capacity (termed Seagate-SMR in the following), and Seagate ST8000AS0002, a similar drive with six platters, 12 heads, and 8TB capacity. Emulated drives use a Seagate ST4000NC001 (Seagate-CMR), a real CMR drive identical in drive mechanics and specification (except the 4TB capacity) to the ST5000AS0011. Results for the 8 and 5TB SMR drives were similar; to save space, we only present results for the publicly available 5TB drive.

4. CHARACTERIZATION TESTS

To motivate our drive characterization methodology we first describe the goals of our measurements. We then describe the mechanisms and methodology for the tests, and finally present results for each tested drive. For emulated SMR drives, we show that the tests produce accurate answers, based on implemented parameters; for real SMR drives we discover their properties. The behavior of the real SMR drives under some of the tests engenders further investigation, leading to the discovery of important details about their operation.

4.1. Characterization Goals

The goal of our measurements is to determine key drive characteristics and parameters:

- Drive type*. In the absence of information from the vendor, is a drive an SMR or a CMR?
- Persistent cache type*. Does the drive use flash or disk for the persistent cache? The type of the persistent cache affects the performance of random writes and reliable (volatile cache-disabled) sequential writes. If the drive uses disk for persistent cache, is it a single cache, or is it distributed across the drive [Cassuto et al. 2010; Hall 2014]? The layout of the persistent disk cache affects the cleaning performance and the performance of the sequential read of a sparsely overwritten linear region.

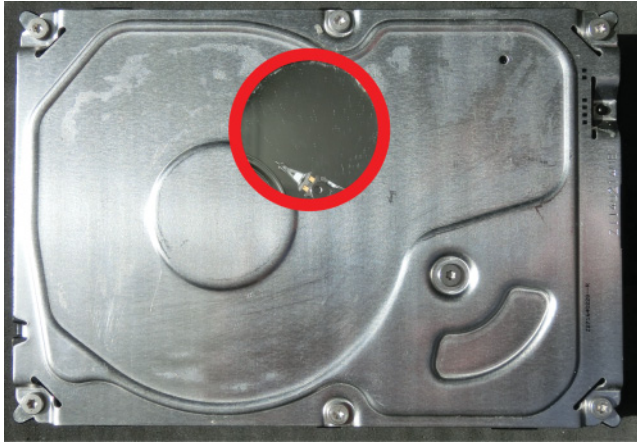


Fig. 3. SMR drive with the observation window encircled in red. Head assembly is visible parked at the inner diameter.

- Cleaning*. Does the drive use aggressive cleaning, improving performance for low duty-cycle applications, or lazy cleaning, which may be better for throughput-oriented ones? Can we predict the performance impact of cleaning?
- Persistent cache size*. After some number of out-of-place writes the drive will need to begin a cleaning process, moving data from the persistent cache to bands so that it can accept new writes, negatively affecting performance. What is this limit, as a function of total blocks written, number of write operations, and other factors?
- Band size*. Since a band is the smallest unit that may be rewritten efficiently, knowledge of band size is important for optimizing SMR drive workloads [Cassuto et al. 2010; Coker and Hall 2013]. What are the band sizes for a drive, and are these sizes constant over time and space [Feldman 2011]?
- Block mapping*. The mapping type affects performance of both cleaning and reliable sequential writes. For LBAs that are not in the persistent cache, is there a static mapping from LBAs to PBAs, or is this mapping dynamic?
- Zone structure*. Determining the zone structure of a drive is a common step in understanding block mapping and band size, although the structure itself has little effect on external performance.

4.2. Test Mechanisms

The software part of Skylight uses `fio` to generate microbenchmarks that elicit the drive characteristics. The hardware part of Skylight tracks the head movement during these tests. It resolves ambiguities when interpreting the latency of the data obtained from the microbenchmarks and leads to discoveries that are not possible with microbenchmarks alone. To track the head movements, we installed (under clean-room conditions) a transparent window in the drive casing over the region traversed by the head. Figure 3 shows the head assembly parked at the Inner Diameter (ID). We recorded the head movements using a Casio EX-ZR500 camera at 1,000 frames per second and processed the recordings with `ffmpeg` to generate head location value for each video frame.

We ran the tests on a 64-bit Intel Core-i3 Haswell system with 16GiB RAM and 64-bit Linux kernel version 3.14. Unless otherwise stated, we disabled kernel read-ahead, drive look-ahead, and drive volatile cache using `hdparm`.

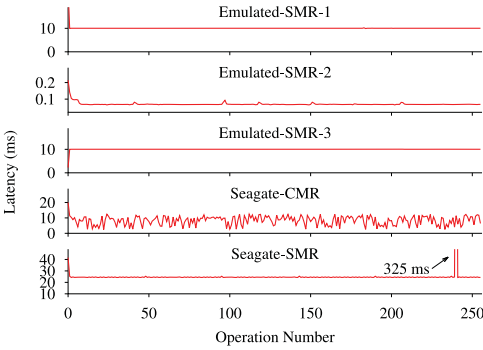


Fig. 4. Discovering drive type using latency of random writes. Y axis varies in each graph.

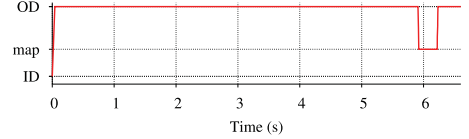


Fig. 5. Seagate-SMR head position during random writes.

Extensions to `fiio` developed for these tests have been integrated back and are available in the latest `fiio` release. Slow-motion clips for the head position graphs shown in the article, as well as the tests themselves, are available at <http://sssl.ccs.neu.edu/skylight>.

4.3. Drive Type and Persistent Cache Type

TEST 1 exploits the unusual random write behavior of the SMR drives to differentiate them from CMR drives. While random writes to a CMR drive incur varying latency due to random seek time and rotational delay, random writes to an SMR drive are sequentially logged to the persistent cache with a fixed latency. If random writes are not local, SMR drives using separate persistent caches by the LBA range [Cassuto et al. 2010] may still incur varying write latency. Therefore, random writes are done within a small region to ensure that a single persistent cache is used.

TEST 1: Discovering Drive Type

- 1 Write blocks in the first 1GiB in random order to the drive.
 - 2 **If** latency is fixed **then** the drive is SMR **else** the drive is CMR.
-

Figure 4 shows the results for this test. Emulated-SMR-1 sequentially writes incoming random writes to the persistent cache. It fills one empty block after another and due to synchronicity of the writes it misses the next empty block by the time the next write arrives. Therefore, it waits for a complete rotation resulting in a 10ms write latency, which is the rotation time of the underlying CMR drive. The submillisecond latency of Emulated-SMR-2 shows that this drive uses flash for the persistent cache. The latency of Emulated-SMR-3 is identical to that of Emulated-SMR-1, suggesting a similar setup. The varying latency of Seagate-CMR identifies it as a conventional drive. Seagate-SMR shows a fixed ≈ 25 ms latency with a ≈ 325 ms bump at the 240th write. While the fixed latency indicates that it is an SMR drive, we resort to the head position graph to understand why it takes 25ms to write a single block and what causes the 325ms latency.

Figure 5 shows that the head, initially parked at the ID, seeks to the Outer Diameter (OD) for the first write. It stays there during the first 239 writes (incidentally, showing that the persistent cache is at the OD), and on the 240th write it seeks to the center, staying there for ≈ 285 ms before seeking back and continuing to write.

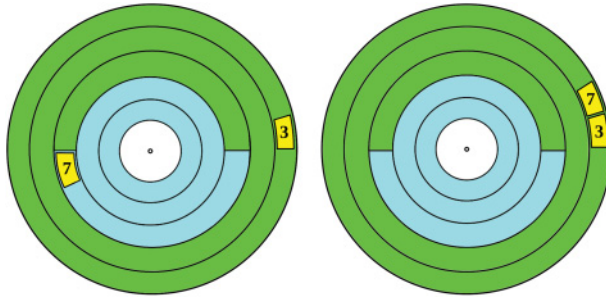


Fig. 6. Surface of a disk platter in a hypothetical SMR drive divided into two 2.5 track imaginary regions. The left figure shows the placement of random blocks 3 and 7 when writing synchronously. Each internal write contains a single block and takes 25ms (50ms in total) to complete. The drive reports 25ms write latency for each block; reading the blocks in the written order results in a 5ms latency. The right figure shows the placement of blocks when writing asynchronously with high queue depth. A single internal write contains both of the blocks, taking 25ms to complete. The drive still reports 25ms write latency for each block; reading the blocks back in the written order results in a 10ms latency due to missed rotation.

Is all of 25ms latency associated with every block write spent writing or is some of it spent in rotational delay? When we repeat the test multiple times, the completion time of the first write ranges between 41 and 52ms, while the remaining writes complete in 25ms. The latency of the first write always consists of a seek from the ID to the OD (≈ 16 ms). We hypothesize that the remaining time is spent in rotational delay—likely waiting for the beginning of a delimited location—and writing (25ms). Depending on where the head lands after the seek, the latency of the first write changes between 41 and 52ms. The remaining writes are written as they arrive, without seek time and rotational delay, each taking 25ms. Hence, we hypothesize that a single block *host write* results in a 2.5 track *internal write*. We realize that 25ms latency is artificially high and expect it to drop in future drives, nevertheless, we base our further explanations on this assumption. In the following section we explore this phenomenon further.

4.3.1. Journal Entries with Quantized Sizes. If after TEST 1 we immediately read blocks in the written order, read latency is fixed at ≈ 5 ms, indicating 0.5 track distance (covering a complete track takes a full rotation, which is 10ms for the drive; therefore, 5ms translates to 0.5 track distance) between blocks. On the other hand, if we write blocks asynchronously at the maximum queue depth of 31 [Libata FAQ 2011] and immediately read them, latency is fixed at ≈ 10 ms, indicating a missed rotation due to contiguous placement. Furthermore, although the drive still reports 25ms completion time for every write, asynchronous writes complete faster—for the 256 write operations, asynchronous writes complete in 216ms, whereas synchronous writes complete in 6,539ms, as seen in Figure 5. Gathering these facts, we arrive at Figure 6. Writing asynchronously with high queue depth allows the drive to pack multiple blocks into a single internal write, placing them contiguously (shown on the right). The drive reports the completion of individual host writes packed into the same internal write once the internal write completes. Thus, although each of the host writes in the same internal write is reported to take 25ms, it is the same 25ms that went into writing the internal write. As a result, in the asynchronous case, the drive does fewer internal writes, which accounts for the fast completion time. The contiguous placement also explains the 10ms latency when reading blocks in the written order. Writing synchronously, however, results in doing a separate internal write for every block (shown on the left), taking longer to complete. Placing blocks starting at the beginning of 2.5 track internal writes explains the 5ms latency when reading blocks in the written order.

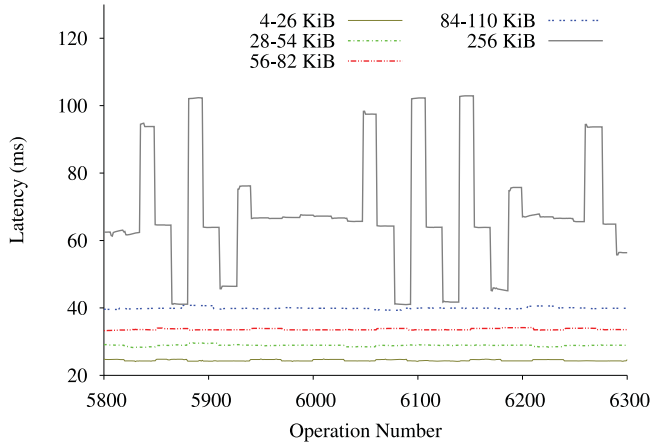


Fig. 7. Random write latency of different write sizes on Seagate-SMR, when writing at the queue depth of 31. Each latency graph corresponds to the latency of a group of writes. For example, the graph at 25ms corresponds to the latency of writes with sizes in the range of 4–26KiB. Since writes with different sizes in a range produced similar latency, we plotted a single latency as a representative.

To understand how the internal write size changes with the increasing host write size, we keep writing at the maximum queue depth, gradually increasing the write size. Figure 7 shows that the writes in the range of 4–26KiB result in 25ms latency, suggesting that 31 host writes in this size range fit in a single internal write. As we jump to the 28KiB writes, the latency increases by ≈ 5 ms (or 0.5 track) and remains approximately constant for the writes of sizes up to 54KiB. We observe a similar jump in latency as we cross from 54 to 56KiB and also from 82 to 84KiB. This shows that the internal write size increases in 0.5 track increments. Given that the persistent cache is written using a “log-structured journaling mechanism” [Feldman 2014a], we infer that the 0.5 track of 2.5 track minimum internal write is the journal entry that grows in 0.5 track increments, and the remaining two tracks contain out-of-band data, like parts of the persistent cache map affected by the host writes. The purpose of this quantization of journal entries is not known, but may be in order to reduce rotational delay or simplify delimiting and locating them. We further hypothesize that the 325ms delay in Figure 4, observed every 240th write, is a map merge operation that stores the updated map at the middle tracks.

As the write size increases to 256KiB we see varying delays, and inspection of completion times shows less than 31 writes completing in each burst, implying a bound on the journal entry size. Different completion times for large writes suggest that for these, the journal entry size is determined dynamically, likely based on the available drive resources at the time when the journal entry is formed.

4.4. Disk Cache Location and Layout

We next determine the location and layout of the disk cache, exploiting a phenomenon called *fragmented reads* [Cassuto et al. 2010]. When sequentially reading a region in an SMR drive, if the cache contains a newer version of some of the blocks in the region, the head has to seek to the persistent cache and back, physically fragmenting a logically sequential read. In TEST 2, we use these variations in seek time to discover the location and layout of the disk cache.

The test works by choosing a small region and writing every other block in it and then reading the region sequentially from the beginning, forcing a fragmented read. LBA numbering conventionally starts at the OD and grows towards the ID. Therefore,

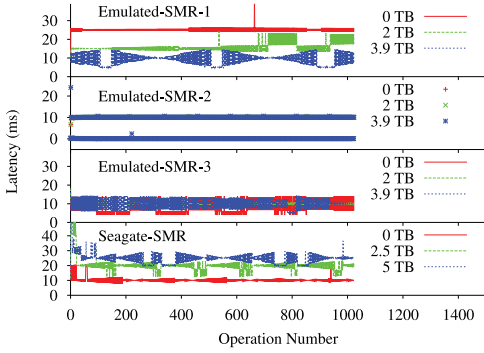


Fig. 8. Discovering disk cache structure and location using fragmented reads.

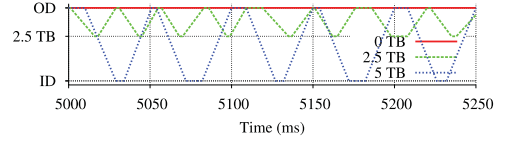


Fig. 9. Seagate-SMR head position during fragmented reads.

TEST 2: Discovering Disk Cache Location and Layout

- 1 Starting at a given offset, write a block and skip a block, and so on, writing 512 blocks in total.
- 2 Starting at the same offset, read 1024 blocks; call average latency lat_{offset} .
- 3 Repeat steps 1 and 2 at the offsets *high*, *low*, *mid*.
- 4 **if** $lat_{high} < lat_{mid} < lat_{low}$ **then**
 | There is a single disk cache at the ID.
else if $lat_{high} > lat_{mid} > lat_{low}$ **then**
 | There is a single disk cache at the OD.
else if $lat_{high} = lat_{mid} = lat_{low}$ **then**
 | There are multiple disk caches.
else
 | **assert**($lat_{high} = lat_{low}$ **and** $lat_{high} > lat_{mid}$);
 | There is a single disk cache in the middle.

a fragmented read at low LBAs on a drive with the disk cache located at the OD would incur negligible seek time, whereas a fragmented read at high LBAs on the same drive would incur high seek time. Conversely, on a drive with the disk cache located at the ID, a fragmented read would incur high seek time at low LBAs and negligible seek time at high LBAs. On a drive with the disk cache located at the Middle Diameter (MD), fragmented reads at low and high LBAs would incur similar high seek times and they would incur negligible seek times at middle LBAs. Finally, on a drive with multiple disk caches evenly distributed across the drive, the fragmented read latency would be mostly due to rotational delay and vary little across the LBA space. Guided by these assumptions, to identify the location of the disk cache, the test chooses a small region at low, middle, and high LBAs and forces fragmented reads at these regions.

Figure 8 shows the latency of fragmented reads at three offsets on all SMR drives. The test correctly identifies the Emulated-SMR-1 as having a single cache at the ID. For Emulated-SMR-2 with flash cache, latency is seen to be negligible for flash reads, and a full missed rotation for each disk read. Emulated-SMR-3 is also correctly identified as having multiple disk caches—the latency graph of all fragmented reads overlap, all having the same 10ms average latency. For Seagate-SMR¹ we confirm that it has a single disk cache at the OD.

¹Test performed with volatile cache enabled with `hdparm -W1`.

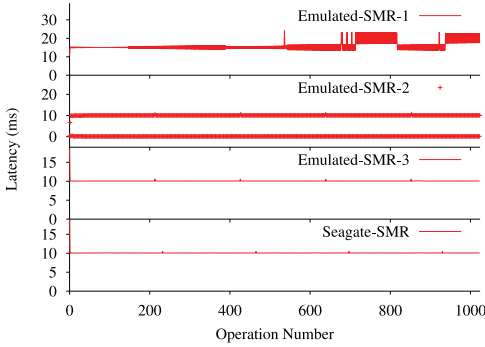


Fig. 10. Discovering cleaning type.

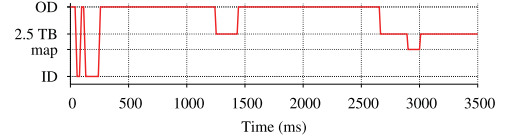


Fig. 11. Seagate-SMR head position during the 3.5-s period starting at the beginning of step 2 of Test Algorithm 3.

Figure 9 shows the Seagate-SMR head position during fragmented reads at offsets of 0, 2.5, and 5TB. For the offsets of 2.5 and 5 TB, we see that the head seeks back and forth between the OD and near-center and between the OD and the ID, respectively, occasionally missing a rotation. The cache-to-data distance for the LBAs near 0TB was too small for the resolution of our camera.

4.5. Cleaning Algorithm

The fragmented read effect is also used in TEST 3 to determine whether the drive uses aggressive or lazy cleaning, by creating a fragmented region and then pausing to allow an aggressive cleaning to run before reading the region.

TEST 3: Discovering Cleaning Type

- 1 Starting at a given offset, write a block and skip a block and so on, writing 512 blocks in total.
 - 2 Pause for 3–5s.
 - 3 Starting at the same offset, read 1,024 blocks.
 - 4 **if** latency is fixed **then** cleaning is aggressive **else** cleaning is lazy.
-

Figure 10 shows the read latency graph of step 3 from TEST 3 at the offset of 2.5TB, with a 3s pause in step 2. For all drives, offsets were chosen to land within a single band (Section 4.8). After a pause, the top two emulated drives continue to show fragmented read behavior, indicating lazy cleaning, while in Emulated-SMR-3 and Seagate-SMR reads are no longer fragmented, indicating aggressive cleaning.

Figure 11 shows the Seagate-SMR head position during the 3.5second period starting at the beginning of step 2. Two short seeks from the OD to the ID and back are seen in the first 200ms; their purpose is not known. The RMW operation for cleaning a band starts at 1,242ms after the last write, when the head seeks to the band at 2.5TB offset, reads for 180ms, and seeks back to the cache at the OD where it spends 1,210ms. We believe this time is spent forming an updated band and persisting it to the disk cache, to protect against power failure during band overwrite. Next, the head seeks to the band, taking 227ms to overwrite it and then seeks to the center to update the map. Hence, cleaning a band in this case took ≈ 1.6 s. We believe the center to contain the map because the head always moves to this position after performing a RMW, and stays there for a short period before eventually parking at the ID. After 3s, reads begin

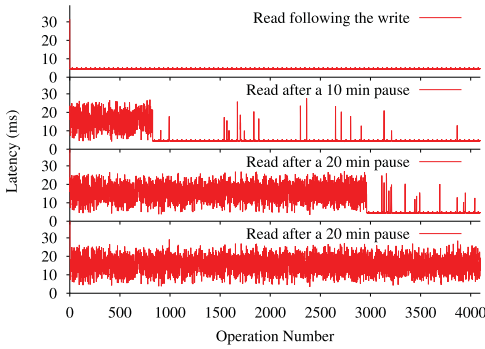


Fig. 12. Latency of reads of random writes immediately after the writes and after 10–20min pauses.

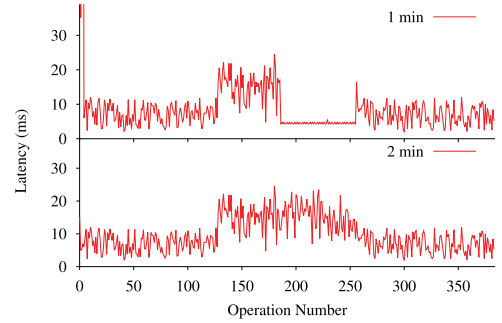


Fig. 13. Verifying hypothesized cleaning algorithm on Seagate-SMR.

and the head seeks back to the band location, where it stays until reads complete (only the first 500ms is seen in Figure 11).

We confirmed that the operation starting at 1,242ms is indeed an RMW: when step 3 is begun before the entire cleaning sequence has completed, read behavior is unchanged from TEST 2. We did not explore the details of the RMW; alternatives like *partial read-modify-write* [Poudyal 2013] may also have been used.

4.5.1. Seagate-SMR Cleaning Algorithm. We next start exploring performance-relevant details that are specific to the Seagate-SMR cleaning algorithm, by running TEST 4. In step 1, as the drive receives random writes, it sequentially logs them to the persistent cache as they arrive. Therefore, immediately reading the blocks back in the written order should result in a fixed rotational delay with no seek time. During the pause in step 3, the cleaning process moves the blocks from the persistent cache to their native locations. As a result, reading after the pause should incur varying seek time and rotational delay for the blocks moved by the cleaning process, whereas unmoved blocks should still incur a fixed latency.

TEST 4: Exploring Cleaning Algorithm

- 1 Write 4096 random blocks.
 - 2 Read back the blocks in the written order.
 - 3 Pause for 10–20min.
 - 4 Repeat steps 2 and 3.
-

In Figure 12, read latency is shown immediately after step 2, and then after 10, 30, and 50min. We observe that the latency is fixed when we read the blocks immediately after the writes. If we reread the blocks after a 10min pause, we observe random latencies for the first ≈ 800 blocks, indicating that the cleaning process has moved these blocks to their native locations. Since every block is expected to be on a different band, the number of operations with random read latencies after each pause shows the progress of the cleaning process, that is, the number of bands it has cleaned. Given that it takes ≈ 30 min to clean $\approx 3,000$ bands, it takes ≈ 600 ms to clean a band whose single block has been overwritten. We also observe a growing number of cleaned blocks in the unprocessed region (for example, operations 3,000–4,000 in the 30-min graph); based on this behavior, we hypothesize that cleaning follows Algorithm 1.

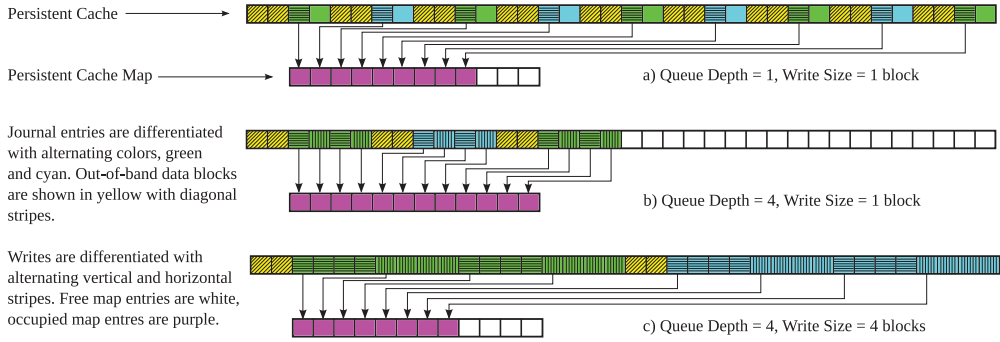


Fig. 14. Three different scenarios triggering cleaning on drives using journal entries with quantized sizes and extent mapping. The text on the left in the figure explains the meaning of the colors.

ALGORITHM 1: Hypothesized Cleaning Algorithm of Seagate-SMR

- 1 Read the next block from the persistent cache, find the block's band.
 - 2 Scan the persistent cache identifying blocks belonging to the band.
 - 3 Read-modify-write the band, update the map.
-

To test this hypothesis we run TEST 5. In Figure 13 we see that after 1min, all of the blocks written in step 1, some of those written in step 2, and all of those written in step 3 have been cleaned, as indicated by the nonuniform latency, while the remainder of step 2 blocks remain in the cache, confirming our hypothesis. After 2min all blocks have been cleaned. (The higher latency for step 2 blocks is due to their higher mean seek distance.)

TEST 5: Verifying the Hypothesized Cleaning Algorithm

- 1 Write 128 blocks from a 256MiB linear region in random order.
 - 2 Write 128 random blocks across the LBA space.
 - 3 Repeat step 1, using different blocks.
 - 4 Pause for 1min; read all blocks in the written order.
-

4.6. Persistent Cache Size

We discover the size of the persistent cache by ensuring that the cache is empty and then measuring how much data may be written before cleaning begins. We use random writes across the LBA space to fill the cache, because sequential writes may fill the drive bypassing the cache [Cassuto et al. 2010] and cleaning may never start. Also, with sequential writes, a drive with multiple caches may fill only one of the caches and start cleaning before all of the caches are full [Cassuto et al. 2010]. With random writes, bypassing the cache is not possible; also, they will fill multiple caches at the same rate and cleaning will start when all of the caches are almost full.

The simple task of filling the cache is complicated in drives using extent mapping: a cache is considered full when the extent map is full or when the disk cache is full, whichever happens first. The latter is further complicated by journal entries with quantized sizes—as seen previously (Section 4.3.1), a single 4KB write may consume as much cache space as dozens of 8KB writes. Due to this overhead, the actual size of the disk cache is larger than what is available to host writes—we differentiate the two by calling them *persistent cache raw size* and *persistent cache size*, respectively.

Figure 14 shows three possible scenarios on a hypothetical drive with a persistent cache raw size of 36 blocks and a 12 entry extent map. The minimum journal entry

size is two blocks, and it grows in units of two blocks to the maximum of 16 blocks; out-of-band data of two blocks is written with every journal entry; the persistent cache size is 32 blocks.

Part (a) of Figure 14 shows the case of queue depth 1 and one-block writes. After the host issues nine writes, the drive puts every write to a separate two-block journal entry, fills the cache with nine journal entries, and starts cleaning. Every write consumes a slot in the map, shown by the arrows. Due to low queue depth, the drive leaves one empty block in each journal entry, wasting nine blocks. Exploiting this behavior, TEST 6 discovers the persistent cache raw size. (In this and the following tests, we detect the start of cleaning when the IOPS drops to near zero.)

TEST 6: Discovering Persistent Cache Raw Size

- 1 Write with a small size and low queue depth until cleaning starts.
 - 2 Persistent cache raw size = number of writes $\times$ (minimum journal entry size + out-of-band data size).
-

Part (b) of Figure 14 shows the case of queue depth 4 and one-block writes. After the host issues 12 writes, the drive forms three four-block journal entries. Writing these journal entries to the cache fills the map and the drive starts cleaning despite a half-empty cache. We use TEST 7 to discover the *persistent cache map size*.

TEST 7: Discovering Persistent Cache Map Size

- 1 Write with a small size and high queue depth until cleaning starts.
 - 2 Persistent cache map size = number of writes.
-

Finally, part (c) of Figure 14 shows the case of queue depth 4 and four-block writes. After the host issues eight writes, the drive forms two 16-block journal entries, filling the cache. Due to high queue depth and large write size, the drive is able to fill the cache (without wasting any blocks) before the map fills. We use TEST 8 to discover the *persistent cache size*.

TEST 8: Discovering Persistent Cache Size

- 1 Write with a large size and high queue depth until cleaning starts.
 - 2 Persistent cache size = total host write size.
-

Table II shows the result of the tests on Seagate-SMR and Figure 15 shows the corresponding graph. In the first row, we discover persistent cache raw size using TEST 6. Writing with 4KiB size and queue depth of 1 produces a fixed 25ms latency (Section 4.3) that is 2.5 rotations. Hypothesizing that all of the 25ms is spent writing and a track size is $\approx 2\text{MiB}$ at the OD, 22,800 operations correspond to $\approx 100\text{GiB}$.

In rows 2 and 3 we discover the persistent cache map size using TEST 7. For write sizes of 4 and 64KiB cleaning starts after $\approx 182,200$ writes, which corresponds to 0.7 and 11.12GiB of host writes, respectively. This confirms that in both cases the drive hits the map size limit, corresponding to scenario (b) in Figure 14. Assuming that the drive uses a low watermark to trigger cleaning, we estimate that the map size is 200,000 entries.

In rows 4 and 5 we discover the persistent cache size using TEST 8. With 128KiB writes we write $\approx 17\text{GiB}$ in fewer operations than in row 3, indicating that we are

Table II. Discovering Persistent Cache Parameters

Drive	Write Size	Queue Depth	Operation Count	Host Writes	Internal Writes
Seagate-SMR	4KiB	1	22,800	89MiB	100GiB^a
	4KiB	31	182,270	0.7GiB	N/A
	64KiB	31	182,231	11.12GiB	N/A
	128KiB	31	137,496	16.78GiB	N/A
	256KiB	31	67,830	16.56GiB	N/A
Emulated-SMR-1	4KiB	1	9,175,056	35GiB	35GiB
Emulated-SMR-2	4KiB	1	2,464,153	9.4GiB	9.4GiB
Emulated-SMR-3	4KiB	1	9,175,056	35GiB	35GiB

^aThis estimate is based on the hypothesis that all of 25ms during a single block write is spent writing to disk. While the results of the experiments indicate this to be the case, we think 25ms latency is artificially high and expect it to drop in future drives, which would require recalculation of this estimate.

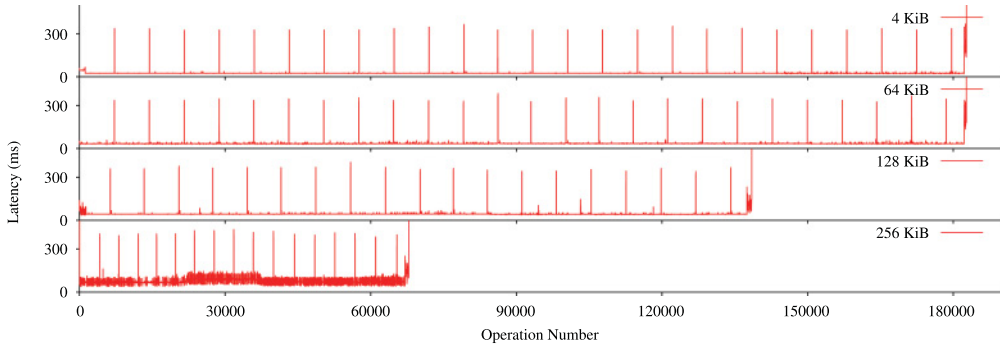


Fig. 15. Write latency of asynchronous writes of varying sizes with queue depth of 31 until cleaning starts. Starting from the top, the graphs correspond to lines 2–5 in Table II. When writing asynchronously, more writes are packed into the same journal entry. Therefore, although the map merge operations still occur at every 240th journal write, the interval seems greater than in Figure 16. For 4 and 64KiB write sizes, we hit the map size limit first, hence cleaning starts after the same number of operations. For 128KiB write size we hit the space limit before hitting the map size limit; therefore, cleaning starts after fewer number of operations than in 64KiB writes. Doubling the write size to 256KiB confirms that we are hitting the space limit, since cleaning starts after half the number of operations of 128KiB writes.

hitting the size limit. To confirm this, we increase write size to 256KiB in row 5; as expected, the number of operations drops by half while the total write size stays the same. Again, assuming that the drive has hit the low watermark, we estimate that the persistent cache size is 20GiB.

Journal entries with quantized sizes and extent mapping are absent topics in academic literature on SMR, so emulated drives implement neither feature. Running TEST 6 on emulated drives produces all three answers, since in these drives, the cache is block-mapped, and the cache size and cache raw size are the same. Furthermore, set-associative STL divides the persistent cache into cache bands and assigns data bands to them using modulo arithmetic. Therefore, despite having a single cache, under random writes it behaves similarly to a fully associative cache. The bottom rows of Table II show that in emulated drives, TEST 8 discovers the cache size (see Table I) with 95% accuracy.

4.7. Is Persistent Cache Shingled?

We next determine whether the STL manages the persistent cache as a circular log. While this would not guarantee that the persistent cache is shingled (an STL could also manage a random-write region as a circular log), it would strongly indicate that

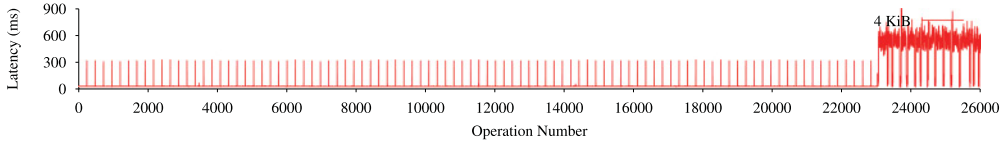


Fig. 16. Write latency of 4KiB synchronous random writes, corresponding to the first line in Table II. As was explained in Section 4.3.1, when writing synchronously the drive writes a journal entry for every write operation. Every 240th journal entry write results in a ≈ 325 ms latency, which as was hypothesized in Section 4.3.1 includes a map merge operation. After $\approx 23,000$ writes, cleaning starts and the IOPS drops precipitously to 0–3. To emphasize the high latency of writes during cleaning we perform 3,000 more operations. As the graph shows, these writes (23,000–26,000) have ≈ 500 ms latency.

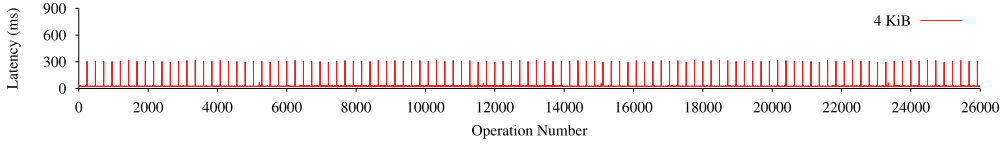


Fig. 17. Write latency of 30,000 random block writes with a repeating pattern. We choose 10,000 random blocks across the LBA space and write them in the chosen order. We then write the same 10,000 blocks in the same order two more times. Unlike Figure 16, the cleaning does not start after $\approx 23,000$ writes, because due to the repeating pattern, as the head of the log wraps around, the STL only finds stale blocks that it can overwrite without cleaning.

the persistent cache is shingled. We start with TEST 9, which chooses a sequence of 10,000 random LBAs across the drive space and writes the sequence straight through, three times. Given that the persistent cache has space for $\approx 23,000$ synchronous block writes (Table II), a dumb STL would fill the cache and start cleaning before the writes complete.

TEST 9: Discovering if the Persistent Cache is a Circular Log—Part I

- 1 Choose 10,000 random blocks across the LBA space.
 - 2 **for** $i \leftarrow 0$ **to** $i < 3$ **do**
 - 3 Write the 10,000 blocks from step 1 in the chosen order.
-

Figure 17 shows that unlike Figure 16, cleaning does not start after 23,000 writes. Two simple hypotheses that explain this phenomenon are as follows:

- (1) The STL manages the persistent cache as a circular log. When the head of the log wraps around, STL detects stale blocks and overwrites without cleaning.
- (2) The STL overwrites blocks in-place. Since there are 10,000 unique blocks, we never fill the persistent cache and cleaning never starts.

To find out which one of these is true, we run TEST 10. Since there are still 10,000 unique blocks, if the hypothesis (2) is true, that is, if the STL overwrites the blocks in-place, we should never consume more than 10,000 writes' worth of space and cleaning should not start before the writes complete. Figure 18 shows that cleaning starts after $\approx 23,000$ writes, invalidating hypothesis (2). Furthermore, if we compare Figure 18 to Figure 16, we see that the latency of writes after cleaning starts is ≈ 100 ms and ≈ 500 ms, respectively. This corroborates hypothesis (1)—latency is lower in the former, because after the head of the log wraps around, the STL finds some stale blocks (since these blocks were chosen from a small pool of 10,000 unique blocks), that it can overwrite without cleaning. When the blocks are chosen across the LBA space, as in Figure 16,

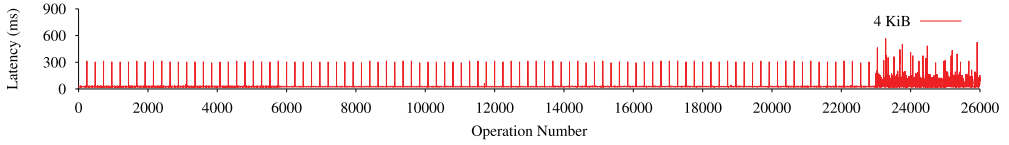


Fig. 18. Write latency of 30,000 random block writes chosen from a pool of 10,000 unique blocks. Unlike Figure 17, cleaning starts after $\approx 23,000$, because as the head of the log wraps around, the STL does not immediately find stale blocks. However, since the blocks are chosen from a small pool, the STL still does find a large number of stale blocks and can often overwrite without cleaning. Therefore, compared to Figure 16 the write latency during cleaning (operations 23,000–26,000) is not as high, since in Figure 16 the blocks are chosen across the LBA space and the STL almost never finds a stale block when the head of the log wraps around.

once the head wraps around, the STL ends up effectively cleaning before every write since it almost never finds a stale block.

TEST 10: Discovering if the Persistent Cache is a Circular Log—Part II

- 1 Choose 10,000 random blocks across the LBA space.
 - 2 **for** $i \leftarrow 0$ **to** $i < 30,000$ **do**
 - 3 Randomly choose a block from the blocks in step 1 and write.
-

4.8. Band Size

STLs proposed to date [Amer et al. 2010; Cassuto et al. 2010; Hall 2014] clean a single band at a time, by reading unmodified data from a band and updates from the cache, merging them, and writing the merge result back to a band. TEST 11 determines the band size, by measuring the granularity at which this cleaning process occurs.

TEST 11: Discovering the Band Size

- 1 Select an accuracy granularity a , and a band size estimate b .
 - 2 Choose a linear region of size $100 \times b$ and divide it into a -sized blocks.
 - 3 Write 4KiB to the beginning of every a -sized block, in random order.
 - 4 Force cleaning to run for a few seconds and read 4KiB from the beginning of every a -sized block in sequential order.
 - 5 Consecutive reads with identical high latency identify a cleaned band.
-

Assuming that the linear region chosen in TEST 11 lies within a region of equal track length, for data that is not in the persistent cache, 4KB reads at a fixed stride a should see identical latencies—that is, a rotational delay equivalent to $(a \bmod T)$ bytes where T is the track length. Conversely, reads of data from cache will see varying delays in the case of a disk cache due to the different (and random) order in which they were written or submillisecond delays in the case of a flash cache.

With aggressive cleaning, after pausing to allow the disk to clean a few bands, a linear read of the written blocks will identify the bands that have been cleaned. For a drive with lazy cleaning the linear region is chosen so that writes fill the persistent cache and force a few bands to be cleaned, which again may be detected by a linear read of the written data.

In Figure 19, we see the results of TEST 11 for $a = 1\text{MiB}$ and $b = 50\text{MiB}$, respectively, with the region located at the 2.5TB offset; for each drive we zoom in to show an individual band that has been cleaned. We correctly identify the band size for the

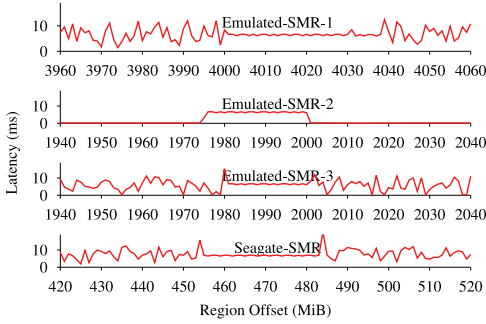


Fig. 19. Discovering band size.

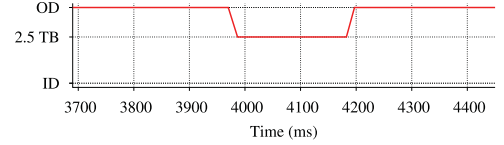


Fig. 20. Head position during the sequential read for Seagate-SMR, corresponding to the time period in Figure 19.

emulated drives (see Table I). The band size of Seagate-SMR at this location is seen to be 30MiB; running tests at different offsets shows that bands are isocapacity within a zone (Section 4.12) but vary from 36MiB at the OD to 17MiB at the ID.

Figure 20 shows the head position of Seagate-SMR corresponding to the time period in Figure 19. It shows that the head remains at the OD during the reads from the persistent cache up to 454MiB, then seeks to 2.5TB offset and stays there for 30MiB, and then seeks back to the cache at OD, confirming that the blocks in the band are read from their native locations.

4.9. Cleaning Time of a Single Band

We observed that cleaning a band whose single block was overwritten can take ≈ 600 ms, whereas if we overwrite 2MiB of the band by skipping every other block, cleaning time increases to ≈ 1.6 s (Section 4.5). While ≈ 600 ms cleaning time due to a single block overwrite gives us a lower bound on the cleaning time, we do not know the upper bound. Now that we understand the persistent cache structure and band size, in addition to the cleaning algorithm, we create an adversarial workload that will give us an upper bound for the cleaning time of a single band.

Table II shows that with a queue depth of 31, we can write 182,270 blocks, that is, 5,880 journal entries, resulting in 700MiB host writes. Assuming the band size is 35MiB at the OD, 700MiB corresponds to 20 bands. Therefore, if we distribute (through random writes) the blocks of 20 bands among 5,880 journal entries, the drive will need to read every packet to clean a single band. Assuming 5–10ms read time for a packet, reading all of the packets to assemble the band will take 29–60s.

To confirm this hypothesis, we shuffled the first 700MiB worth of blocks and wrote them with a queue depth of 31. The cleaning took ≈ 15 min, which is ≈ 45 s per band.

4.10. Block Mapping

Once we discover the band size (Section 4.8), we can use TEST 12 to determine the mapping type. This test exploits varying intertrack switching latency between different track pairs to detect if a band was remapped. After overwriting the first two tracks of band b , cleaning will move the band to its new location—a different physical location only if dynamic mapping is used. Plotting latency graphs of step 2 and step 4 will produce the same pattern for the static mapping and a different pattern for the dynamic mapping.

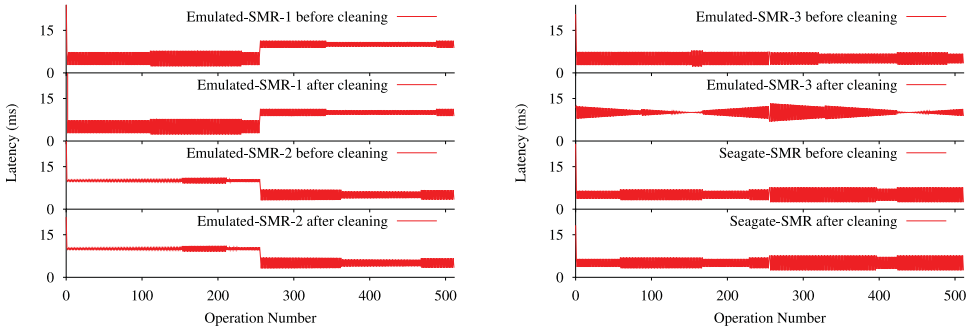


Fig. 21. Detecting mapping type.

TEST 12: Discovering Mapping Type

- 1 Choose two adjacent isocapacity bands a and b ; set n to the number of blocks in a track.
- 2 **for** $i \leftarrow 0$ **to** $i < 2$ **do**
- 3 **for** $j \leftarrow 0$ **to** $j < n$ **do**
 - Read block j of track 0 of band a
 - Read block j of track i of band b
- 4 Overwrite the first two tracks of band b ; force cleaning to run.
- 5 Repeat step 2.

Adapting this test to a drive with lazy cleaning involves some extra work. First, we should start the test on a drive after a secure erase, so that the persistent cache is empty. Due to lazy cleaning, the graph of step 4 will be the graph of switching between a track and the persistent cache. Therefore, we will fill the cache until cleaning starts, and repeat step 2 once in a while, comparing its graph to the previous two: if it is similar to the last, then data is still in the cache; if it is similar to the first, then the drive uses static mapping; otherwise, the drive uses dynamic mapping.

We used track and block terms to concisely describe the preceding test, but the size chosen for these parameters of the test need not match track size and block size of the underlying drive. Figure 21, for example, shows the plots for the test on all of the drives using 2MiB for the track size and 16KiB for the block size. The latency pattern before and after cleaning is different only for Emulated-SMR-3 (seen on the top right), correctly indicating that it uses dynamic mapping. For all of the remaining drives, including Seagate-SMR, the latency pattern is the same before and after cleaning, indicating a static mapping.

4.11. Effect of Mapping Type on Drive Reliability

The type of band mapping used in an SMR drive affects the drive reliability for the reasons explained next. When we enable the volatile cache on Seagate-SMR, it sustains full throughput for sequential writes. Seagate-SMR does not contain flash and it uses static mapping, therefore it can achieve full throughput only if it buffers the data in the volatile cache and writes directly to the band, bypassing the persistent cache.

This performance improvement, however, comes with a risk of data loss. Since there is no backup of the overwritten data, if power is lost midway through the band overwrite, blocks in the following tracks are left in a corrupt state, resulting in data loss. We also lose the new data since it was buffered in the volatile cache.

A similar error, known as *torn write* [Bairavasundaram et al. 2008; Krioukov et al. 2008], occurs in CMR drives as well, wherein only a portion of a sector gets written



Fig. 22. Torn write scenarios in a hypothetical SMR drive with bands consisting of three tracks and the write head with of 1.5 tracks. The tracks are shown horizontally instead of circularly to make the illustration clear. (1) shows the logical view of the band consisting of three tracks, each track having 4,000 sectors. (2) shows the physical layout of the tracks on a platter that accounts for the track skew. (3a) and (3b) show the corruption scenario when the power is lost during the track switch. The red region in (3a) shows the situation after track 1 of the band has been overwritten—track 1 contains new data, whereas track 2 is corrupted; track 3 contains the old data. (3b) shows the situation after track 2 has been overwritten—track 1 and track 2 contain new data, whereas track 3 is corrupted. If power is lost while the head switches from track 2 to track 3, block ranges 10,000–11,999 and 8,000–9,999, or the single range of 8,000–11,999 is left in a corrupt state. (4a) and (4b) show the corruption scenario when the power is lost during the track overwrite. (4a) is identical to (3a) and shows the situation after track 1 of the band has been overwritten. (4b) shows the situation where power is lost after blocks 4,000–4,999 have been overwritten. In this case, block ranges 7,000–7,999, 5,000–6,999, and 11,000–11,999, or two ranges of 5,000–7,999 and 11,000–11,999 are left in a corrupt state.

before power is lost. In CMR drives, the time required to atomically overwrite a sector is small enough that reports of such errors are rare [Krioukov et al. 2008]. An SMR drive with static mapping, on the other hand, is similar to a CMR drive with large (in the case of Seagate-SMR, 17–3 MiB) sectors. Therefore, there is a high probability that a random power loss during streaming sequential writes will disrupt a band overwrite.

Figure 22 describes two error scenarios in a hypothetical SMR drive that uses static mapping. These errors are the consequence of the used mapping scheme, since the only way of sustaining full throughput in such a scheme is to write to the band directly. Introducing a small amount of flash to an SMR drive for persistent buffering has its own challenges—exploiting parallelism for fast flash writes and managing wear leveling is possible only if large amounts of flash are used, which is not feasible inside an SMR drive. On the other hand, when using a dynamic band mapping scheme, similar to fully associative STL, a drive can write the new contents of a band directly to a free band without jeopardizing the existing band data. This, followed by an atomic switch in the mapping table would result in full-throughput sequential writes without sacrificing reliability. The idea is similar to log-block FTLs [Kim et al. 2002; Park et al. 2008] that have been successful in overcoming slow block overwrites in NAND flash. For the reasons described, we expect that the next generation of SMR drives will use dynamic band mapping.

We successfully reproduced torn writes on Seagate-SMR by using an Arduino UNO R3 board with a two-channel relay shield to control the power to the drive. After Running TEST 13 at arbitrary offsets, we could reproduce hard read errors as shown in Figure 23 on all of our sample drives. The offset where errors occurred differed between drives. These errors disappeared after overwriting the affected regions.

```

ata2.00: exception Emask 0x0 SAct 0x1000 SErr 0x0 action 0x0
ata2.00: irq_stat 0x40000008
ata2.00: failed command: READ FPDMA QUEUED
ata2.00: cmd 60/c0:60:40:1d:19/02:00:80:00/40 tag 12 ncq 360448 in
        res 41/00:c0:f8:1e:19/00:02:80:00/00/00 Emask 0x401 (device error) <F>
ata2.00: status: { DRDY ERR }

```

Fig. 23. Hard read error under Linux kernel 3.16 when reading a region affected by a torn write.

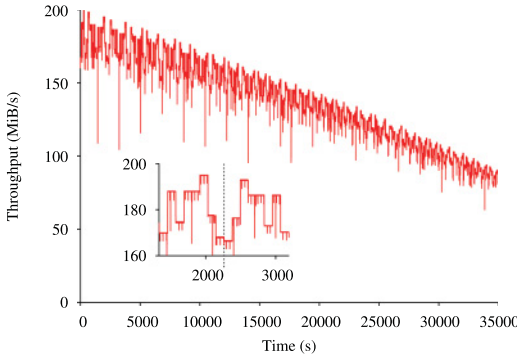


Fig. 24. Sequential read throughput of Seagate-SMR.

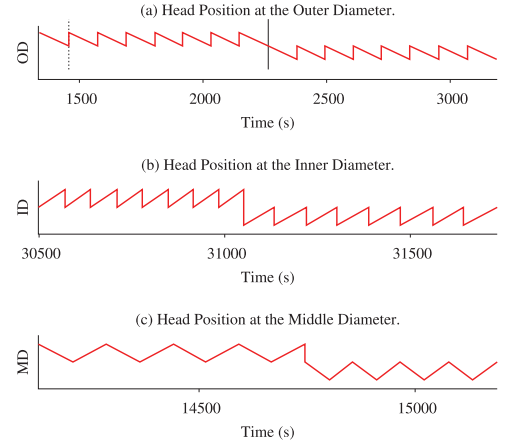


Fig. 25. Seagate-SMR head position during sequential reads at different offsets.

TEST 13: Reproducing Torn Writes

- 1 Choose an *offset*.
 - 2 **for** $i \leftarrow 0$ **to** $i < 50$ **do**
 - 3 Power on the drive and start 1MiB sequential writes at *offset*.
 - 4 After 10s power off the drive; wait for 5s and power on the drive.
 - 5 Starting at the crash point, go back 5000 blocks and read 6000 blocks.
-

4.12. Zone Structure

We use sequential reads (TEST 14) to discover the zone structure of Seagate-SMR. While there are no such drives yet, on drives with dynamic mapping a secure erase that would restore the mapping to the default state may be necessary for this test to work. Figure 24 shows the zone profile of Seagate-SMR, with a zoom to the beginning.

TEST 14: Discovering Zone Structure

- 1 Enable kernel read-ahead and drive look-ahead.
 - 2 Sequentially read the whole drive in 1MiB blocks.
-

Similar to CMR drives, the throughput falls as we reach higher LBAs; unlike CMR drives, there is a pattern that repeats throughout the graph, shown by the zoomed part. This pattern has an axis of symmetry indicated by the dotted vertical line at 2,264th second. There are eight distinct plateaus to the left and to the right of the axis with similar throughputs. The fixed throughput in a single plateau and a sharp change in throughput between plateaus suggest a wide radial stroke and a head switch.

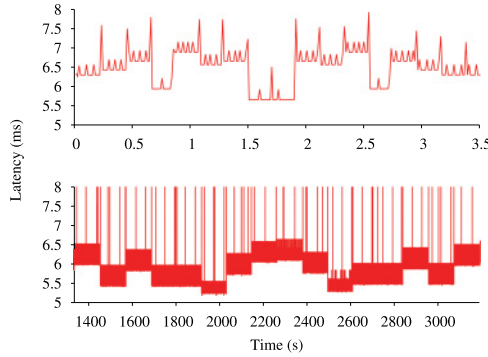


Fig. 26. Sequential read latency of Seagate-CMR (top) and Seagate-SMR (bottom) corresponding to a complete cycle of ascent and descent through platter surfaces. While Seagate-CMR completes the cycle in 3.5s, Seagate-SMR completes it in 1800s, since the latter reads thousands of tracks from a single surface before switching to the next surface.

Plateaus correspond to large zones of size 18–20GiB, gradually decreasing to 4GiB as we approach higher LBAs. The slight decrease in throughput in symmetric plateaus on the right is due to moving from a larger to smaller radii, where sector per track count decreases; therefore, throughput decreases as well.

We confirmed these hypotheses using the head position graph shown in Figure 25(a), which corresponds to the time interval of the zoomed graph of Figure 24. Unlike with CMR drives, where we could not observe head switches due to narrow radial strokes, with this SMR drive head switches are visible to an unaided eye. Figure 25(a) shows that the head starts at the OD and slowly moves toward the MD completing this inwards move at the 1,457th second, indicated by the vertical dotted line. At this point, the head has just completed a wide radial stroke reading gigabytes from the top surface of the first platter, and it performs a jump back to the OD and starts a similar stroke on the bottom surface of the first platter. The direction of the head movement indicates that the shingling direction is toward the ID at the OD. The head completes the descent through the platters at the 2,264th second—indicated by the vertical solid line—and starts its ascent reading surfaces in the reverse order. These wide radial strokes create “horizontal zones” that consist of thousands of tracks on the same surface, as opposed to “vertical zones” spanning multiple platters in CMR drives. We expect these horizontal zones to be the norm in SMR drives, since they facilitate SMR mechanisms like allocation of isocapacity bands, static mapping, and dynamic band size adjustment [Feldman 2011]. Figure 25(b) corresponds to the end of Figure 24, and shows that the direction of the head movement is reversed at the ID, indicating that both at the OD and at the ID, shingling direction is toward the middle diameter. To our surprise, Figure 25(c) shows that a conventional serpentine layout with wide serpents is used at the MD. We speculate that although the whole surface is managed as if it is shingled, there is a large region in the middle that is not shingled.

It is hard to confirm the shingling direction without observing the head movement. The existence of “horizontal zones,” on the other hand, can also be confirmed by contrasting the sequential latency graphs of Seagate-SMR and Seagate-CMR. The bottom of Figure 26 shows the latency graph for the zoomed region in Figure 24. As expected, the shape of the latency graph matches the shape of the throughput graph mirrored around the x axis. The top of Figure 26 shows an excerpt from the latency graph of Seagate-CMR that is also repeated throughout the latency graph. This graph too has a pattern that is mirrored at the center, also indicating a completed ascent and

descent through the surfaces. However, Seagate-CMR completes the cycle in 3.5s since it reads only a few tracks from each surface, whereas Seagate-SMR completes the cycle in 1800s, indicating that it reads thousands of tracks from a single surface.

Smaller spikes at the graph of Seagate-CMR correspond to track switches, and higher spikes correspond to head switches. While the extra 1ms head switch latency every few megabytes does not affect the accuracy of emulation, it shows up in some of the tests, for example, as the bump around 4030thMiB in Figure 19. Figure 26 also shows that the number of platters can be inferred from the latency graph of sequential reads.

5. RELATED WORK

Little has been published on the subject of system-level behavior of SMR drives. Although several works (for example, Amer et al. [2010] and Le et al. [2011]) have discussed requirements and possibilities for use of shingled drives in systems, only three papers to date—Cassuto et al. [2010], Lin et al. [2012], and Hall et al. [2012]—present example translation layers and simulation results. A range of STL approaches is found in the patent literature [Coker and Hall 2013; Fallone and Boyle 2013; Feldman 2011; Hall 2014], but evaluation and analysis is lacking. Several SMR-specific file systems have been proposed, such as SMRfs [Gibson and Polte 2009], SFS [Le Moal et al. 2012], and HiSMRfs [Jin et al. 2014]. He and Du [2014] propose a static mapping to minimize rewrites for in-place updates, which requires high guard overhead (20%) and assumes file system free space is contiguous in the upper LBA region. Pitchumani et al. [2012] present an emulator implemented as a Linux device mapper target that mimics shingled writing on top of a CMR drive. Tan et al. [2013] describe a simulation of S-blocks algorithm, with a more accurate simulator calibrated with data from a real CMR drive. To date, no work (to the authors' knowledge) has presented measurements of read and write operations on an SMR drive, or performance-accurate emulation of STLs.

This work draws heavily on earlier disk characterization studies that have used microbenchmarks to elicit details of internal performance, such as Schlosser et al. [2005], Gim and Won [2010], Krevat et al. [2011], Talagala et al. [1999], and Worthington et al. [1995]. Due to the presence of a translation layer, however, the specific parameters examined in this work (and the microbenchmarks for measuring them) are different.

6. CONCLUSIONS AND RECOMMENDATIONS

As Table III shows, the Skylight methodology enables us to discover key properties of two drive-managed SMR disks automatically. With manual intervention, it allows us to completely reverse engineer a drive. The purpose of doing so is not just to satisfy our curiosity, however, but to guide both their use and evolution. In particular, we draw the following conclusions from our measurements of the 5TB Seagate drive:

- Write latency with the volatile cache disabled is high (TEST 1). This appears to be an artifact of specific design choices rather than fundamental requirements, and we hope for it to drop in later firmware revisions.
- Sequential throughput (with the volatile cache disabled) is much lower (by $3\times$ or more, depending on write size) than for conventional drives. (We omitted these test results, as performance is identical to the random writes in TEST 1.) Due to the use of static mapping (TEST 12), achieving full sequential throughput requires enabling volatile cache.
- Random I/O throughput (with the volatile cache enabled or with high queue depth) is high (TEST 7)— $15\times$ that of the equivalent CMR drive. This is a general property of any SMR drive using a persistent cache.

Table III. Properties of the 5 and the 8TB Seagate Drives Discovered using Skylight Methodology

Property	Drive Model	
	ST5000AS0011	ST8000AS0002 ^a
Drive Type	SMR	SMR
Persistent Cache Type	Disk	Disk
Cache Layout and Location	Single, at the OD	Single, at the OD
Usable Cache Size	20GiB	25GiB
Cache Map Size	200,000	250,000
Band Size	17–36MiB	15–40MiB
Block Mapping	Static	Static
Cleaning Type	Aggressive	Aggressive
Cleaning Algorithm	FIFO	FIFO
Cleaning Time	0.6–45s/band	0.6–45s/band
Zone Structure	4–20GiB	5–40GiB
Shingling Direction	Toward MD	N/A

^aThe benchmarks worked out of the box on the 8TB drive. Since the 8TB drive was on loan, we did not drill a hole on it; therefore, shingling direction for it is not available.

- Throughput may degrade precipitously when the cache fills after many writes (Table II). The point at which this occurs depends on write size and queue depth.²
- Background cleaning begins after ≈ 1 s of idle time, and proceeds in steps requiring 0.6–45s of idle time to clean a single band (Section 4.9).
- Sequential reads of randomly-written data will result in random-like read performance until cleaning completes (Section 4.4).

In summary, SMR drives like the ones we studied should offer good performance if the following conditions are met: (a) the volatile cache is enabled or a high queue depth is used, (b) writes display strong spatial locality, modifying only a few bands at any particular time, (c) nonsequential writes (or all writes, if the volatile cache is disabled) occur in bursts of less than 16GB or 180,000 operations (Table II), and (d) long powered-on idle periods are available for background cleaning. From the use of aggressive cleaning that presumes long idle periods, we may conclude that the drive is adapted to desktop use, but may perform poorly on server workloads. Further work will include investigation of STL algorithms that may offer a better balance of performance for both.

ACKNOWLEDGMENTS

The authors would like to thank the anonymous reviewers, Kimberly Keeton, Tim Feldman, and Remzi Arpaci-Dusseau for their feedback on the FAST version of the article.

REFERENCES

- Ahmed Amer, Darrell D. E. Long, Ethan L. Miller, Jehan-Francois Paris, and S. J. Thomas Schwarz. 2010. Design issues for a shingled write disk system. In *Proceedings of the 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST) (MSST'10)*. IEEE Computer Society, Washington, DC, 1–12. DOI: <http://dx.doi.org/10.1109/MSST.2010.5496991>
- Jens Axboe. 2015. Flexible I/O Tester. [git://git.kernel.dk/fio.git](http://git.kernel.dk/fio.git).
- Lakshmi N. Bairavasundaram, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, Garth R. Goodson, and Bianca Schroeder. 2008. An analysis of data corruption in the storage stack. *Trans. Storage* 4, 3, Article 8 (Nov. 2008), 28 pages. DOI: <http://dx.doi.org/10.1145/1416944.1416947>

²Although results with the volatile cache enabled are not presented in Section 4.6, they are similar to those for a queue depth of 31.

- Luc Bouganim, Bjorn Jnsson, and Philippe Bonnet. 2009. uFLIP: Understanding flash IO patterns. In *Proceedings of the International Conference on Innovative Data Systems Research (CIDR)*. Asilomar, California.
- Yuval Cassuto, Marco A. A. Sanvido, Cyril Guyot, David R. Hall, and Zvonimir Z. Bandic. 2010. Indirection systems for shingled-recording disk drives. In *Proceedings of the 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST) (MSST'10)*. IEEE Computer Society, Washington, DC, 1–14. DOI: <http://dx.doi.org/10.1109/MSST.2010.5496971>
- Feng Chen, David A. Koufaty, and Xiaodong Zhang. 2009. Understanding intrinsic characteristics and system implications of flash memory based solid state drives. In *Proceedings of the 11th International Joint Conference on Measurement and Modeling of Computer Systems (SIGMETRICS'09)*. ACM, New York, NY, 181–192. DOI: <http://dx.doi.org/10.1145/1555349.1555371>
- Jonathan Darrel Coker and David Robison Hall. 2013. Indirection memory architecture with reduced memory requirements for shingled magnetic recording devices. (Nov. 5, 2013). US Patent 8,578,122.
- Linux Device-Mapper. 2001. Device-Mapper Resource Page. <https://sourceware.org/dm/>.
- Elizabeth A. Dobisz, Z. Z. Bandic, Tsai-Wei Wu, and T. Albrecht. 2008. Patterned media: Nanofabrication challenges of future disk drives. *Proc. IEEE* 96, 11 (Nov. 2008), 1836–1846. DOI: <http://dx.doi.org/10.1109/JPROC.2008.2007600>
- DRAMeXchange. 2014. NAND Flash Spot Price. (Sept. 2014). <http://dramexchange.com>.
- Robert M. Fallone and William B. Boyle. 2013. Data storage device employing a run-length mapping table and a single address mapping table. (May 14, 2013). US Patent 8,443,167.
- Tim Feldman. 2014a. Host-aware SMR. OpenZFS Developer Summit. Available from <https://www.youtube.com/watch?v=b1yqjV8qemU>.
- Tim Feldman. 2014b. Personal communication. (Aug. 2014).
- Tim Feldman and Garth Gibson. 2013. Shingled magnetic recording: Areal density increase requires new data management. *USENIX* 38, 3 (2013).
- Timothy Richard Feldman. 2011. Dynamic storage regions. (Feb. 14, 2011). US Patent Appl. 13/026,535.
- Garth Gibson and Greg Ganger. 2011. *Principles of Operation for Shingled Disk Devices*. Technical Report CMU-PDL-11-107. CMU Parallel Data Laboratory. <http://repository.cmu.edu/pdl/7>.
- Garth Gibson and Milo Polte. 2009. *Directions for Shingled-Write and Two-Dimensional Magnetic Recording System Architectures: Synergies with Solid-State Disks*. Technical Report CMU-PDL-09-104. CMU Parallel Data Laboratory. <http://repository.cmu.edu/pdl/7>.
- Jongmin Gim and Youjip Won. 2010. Extract and infer quickly: Obtaining sector geometry of modern hard disk drives. *ACM Trans Storage (TOS)* 6, 2, Article 6 (July 2010), 26 pages. DOI: <http://dx.doi.org/10.1145/1807060.1807063>
- David Hall, John H. Marcos, and Jonathan D. Coker. 2012. Data handling algorithms for autonomous shingled magnetic recording hdds. *IEEE Trans Magn* 48, 5, 1777–1781.
- David Robison Hall. 2014. Shingle-written magnetic recording (SMR) device with hybrid E-region. (April 1, 2014). US Patent 8,687,303.
- Weiping He and David H. C. Du. 2014. Novel address mappings for shingled write disks. In *Proceedings of the 6th USENIX Conference on Hot Topics in Storage and File Systems (HotStorage'14)*. USENIX Association, Berkeley, CA, 5–5. <http://dl.acm.org/citation.cfm?id=2696578.2696583>
- HGST. 2014. HGST Unveils Intelligent, Dynamic Storage Solutions to Transform the Data Center. (Sept. 2014). Available from <http://www.hgst.com/press-room/>.
- INCITS T10 Technical Committee. 2014. *Information technology—Zoned Block Commands (ZBC)*. Draft Standard T10/BSR INCITS 536. American National Standards Institute, Inc. Available from <http://www.t10.org/drafts.htm>.
- Chao Jin, Wei-Ya Xi, Zhi-Yong Ching, Feng Huo, and Chun-Teck Lim. 2014. HiSMRfs: A high performance file system for shingled storage array. In *Proceedings of the 2014 IEEE 30th Symposium on Mass Storage Systems and Technologies (MSST)*. 1–6. DOI: <http://dx.doi.org/10.1109/MSST.2014.6855539>
- Jesung Kim, Jong Min Kim, S. H. Noh, Sang Lyul Min, and Yookun Cho. 2002. A space-efficient flash translation layer for CompactFlash systems. *IEEE Trans. Consumer Electron.* 48, 2 (May 2002), 366–375. DOI: <http://dx.doi.org/10.1109/TCE.2002.1010143>
- Elie Krevat, Joseph Tucek, and Gregory R. Ganger. 2011. Disks are like snowflakes: No two are alike. In *Proceedings of the 13th USENIX Conference on Hot Topics in Operating Systems (HotOS'13)*. USENIX Association, Berkeley, CA, 14–14. <http://dl.acm.org/citation.cfm?id=1991596.1991615>
- Andrew Krioukov, Lakshmi N. Bairavasundaram, Garth R. Goodson, Kiran Srinivasan, Randy Thelen, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2008. Parity lost and parity regained. In

- Proceedings of the 6th USENIX Conference on File and Storage Technologies (FAST'08)*. USENIX Association, Berkeley, CA, Article 9, 15 pages. <http://dl.acm.org/citation.cfm?id=1364813.1364822>
- Mark H. Kryder, Edward C. Gage, Terry W. McDaniel, William A. Challener, Robert E. Rottmayer, Ganping Ju, Yiao-Tee Hsia, and M. Fatih Erden. 2008. Heat assisted magnetic recording. *Proc. IEEE* 96, 11 (Nov. 2008), 1810–1835. DOI: <http://dx.doi.org/10.1109/JPROC.2008.2004315>
- Quoc M. Le, Kumar Sathyanarayana Raju, Ahmed Amer, and JoAnne Holliday. 2011. Workload impact on shingled write disks: All-writes can be alright. In *Proceedings of the 2011 IEEE 19th Annual International Symposium on Modelling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS'11)*. IEEE Computer Society, Washington, DC, 444–446. DOI: <http://dx.doi.org/10.1109/MASCOTS.2011.58>
- Damien Le Moal, Zvonimir Bandic, and Cyril Guyot. 2012. Shingled file system host-side management of Shingled magnetic recording disks. In *Proceedings of the 2012 IEEE International Conference on Consumer Electronics (ICCE)*. 425–426. DOI: <http://dx.doi.org/10.1109/ICCE.2012.6161799>
- Libata FAQ. 2011. https://ata.wiki.kernel.org/index.php/Libata_FAQ.
- Chung-I Lin, Dongchul Park, Weiping He, and David H. C. Du. 2012. H-SWD: Incorporating hot data identification into Shingled write disks. In *Proceedings of the 2012 IEEE 20th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS'12)*. IEEE Computer Society, Washington, DC, 321–330. DOI: <http://dx.doi.org/10.1109/MASCOTS.2012.44>
- Chanik Park, Wonmoon Cheon, Jeonguk Kang, Kangho Roh, Wonhee Cho, and Jin-Soo Kim. 2008. A reconfigurable FTL (Flash Translation Layer) architecture for NAND flash-based applications. *ACM Trans. Embed. Comput. Syst.* 7, 4, Article 38 (Aug. 2008), 23 pages. DOI: <http://dx.doi.org/10.1145/1376804.1376806>
- S. N. Piramanayagam. 2007. Perpendicular recording media for hard disk drives. *J. Appl. Phys.* 102, 1 (July 2007), 011301. DOI: <http://dx.doi.org/10.1063/1.2750414>
- Rekha Pitchumani, Andy Hospodor, Ahmed Amer, Yangwook Kang, Ethan L. Miller, and Darrell D. E. Long. 2012. Emulating a Shingled write disk. In *Proceedings of the 2012 IEEE 20th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS'12)*. IEEE Computer Society, Washington, DC, 339–346. DOI: <http://dx.doi.org/10.1109/MASCOTS.2012.46>
- Sundar Poudyal. 2013. Partial write system. (March 13, 2013). US Patent Appl. 13/799,827.
- Drew Riley. 2013. Samsung's SSD Global Summit: Samsung: Flexing Its Dominance In The NAND Market. (Aug. 2013). <http://www.tomshardware.com/reviews/samsung-global-ssd-summit-2013,3570.html>.
- Mendel Rosenblum and John K. Ousterhout. 1991. The design and implementation of a log-structured file system. In *Proceedings of the 13th ACM Symposium on Operating Systems Principles (SOSP'91)*. ACM, New York, NY, 1–15. DOI: <http://dx.doi.org/10.1145/121132.121137>
- SATA-IO. 2011. *Serial ATA Revision 3.1 Specification*. Technical Report. SATA-IO.
- Steven W. Schlosser, Jiri Schindler, Stratos Papadomanolakis, Minglong Shao, Anastassia Ailamaki, Christos Faloutsos, and Gregory R. Ganger. 2005. On multidimensional data and modern disks. In *Proceedings of the 4th Conference on USENIX Conference on File and Storage Technologies—Volume 4 (FAST'05)*. USENIX Association, Berkeley, CA, 17–17. <http://dl.acm.org/citation.cfm?id=1251028.1251045>
- Seagate 2013a. *Seagate Desktop HDD: ST5000DM000, ST4000DM001*. Product Manual 100743772. Seagate Technology LLC.
- Seagate 2013b. Seagate Technology PLC Fiscal Fourth Quarter and Year End 2013 Financial Results Supplemental Commentary. (July 2013). Available from <http://www.seagate.com/investors>.
- Seagate 2013c. *Terascale HDD*. Data sheet DS1793.1-1306US. Seagate Technology PLC.
- Seagate 2014. Seagate Ships Worlds First 8TB Hard Drives. (Aug. 2014). Available from <http://www.seagate.com/about/newsroom/>.
- Nisha Talagala, Remzi H. Arpaci-Dusseau, and David Patterson. 1999. *Microbenchmark-based Extraction of Local and Global Disk Characteristics*. Technical Report UCB/CSD-99-1063. EECS Department, University of California, Berkeley. <http://www.eecs.berkeley.edu/Pubs/TechRpts/1999/6275.html>.
- Sophia Tan, Weiya Xi, Zhi Yong Ching, Chao Jin, and Chun Teck Lim. 2013. Simulation for a Shingled magnetic recording disk. *IEEE Trans. Magn.* 49, 6 (June 2013), 2677–2681. DOI: <http://dx.doi.org/10.1109/TMAG.2013.2245872>
- David A. Thompson and John S. Best. 2000. The future of magnetic data storage technology. *IBM J. Res. Dev.* 44, 3 (May 2000), 311–322. DOI: <http://dx.doi.org/10.1147/rd.443.0311>
- Sumei Wang, Yao Wang, and Randall H. Victora. 2013. Shingled magnetic recording on bit patterned media at 10 Tb/in². *IEEE Trans. Magn.* 49, 7 (July 2013), 3644–3647. DOI: <http://dx.doi.org/10.1109/TMAG.2012.2237545>

- R. Wood, Mason Williams, A Kavcic, and Jim Miles. 2009. The feasibility of magnetic recording at 10 terabits per square inch on conventional media. *IEEE Trans. Magn.* 45, 2 (Feb. 2009), 917–923. DOI: <http://dx.doi.org/10.1109/TMAG.2008.2010676>
- Bruce L. Worthington, Gregory R. Ganger, Yale N. Patt, and John Wilkes. 1995. On-line extraction of SCSI disk drive parameters. In *Proceedings of the 1995 ACM SIGMETRICS Joint International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS'95/PERFORMANCE'95)*. ACM, New York, NY, 146–156. DOI: <http://dx.doi.org/10.1145/223587.223604>

Received June 2015; revised August 2015; accepted September 2015